

Model-based Software Engineering

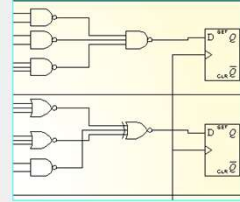
師大資工
鄭永斌

1

Prerequisite (2)

- In E.E., they always simulate their design
- first before manufacturing

VHDL



```

TYPE state_type IS (s0,s1,s2,s3);
SIGNAL present_state,next_state : state_type;

BEGIN
  --state register process
  state_reg:PROCESS
  BEGIN
    WAIT UNTIL rising_edge(clock);
    present_state <= next_state;
  END PROCESS;
  --combinational logic feedback process
  fb_logic:PROCESS(present_state,x)
  BEGIN
    CASE present_state IS
      WHEN s0 =>
        IF x = '0' THEN z <= '0'; next_state <= s0;
        ELSE z <= '1'; next_state <= s2;
        END IF;
      WHEN s1 =>
        IF x = '0' THEN z <= '0'; next_state <= s0;
        ELSE z <= '0'; next_state <= s2;
    END CASE;
  END PROCESS;

```

4

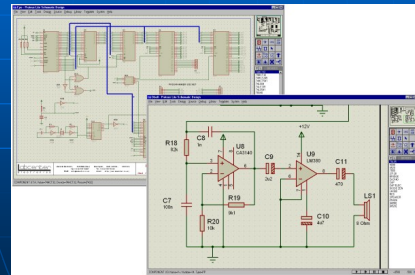
History

- While dealing with complex entity, other engineering has learned not to learn it by building it.
- They approach the complexity cautiously
- Building prototypes and models are two major approaches for tackling with something we do not understand much

2

Prerequisite (3)

Software tools/environment for simulation and verification of hardware design



5

Prerequisite (1)

- In other engineering fields
 - In aircraft manufacturing, engineering build model to test air dynamics in a wind tunnel



NASA's cryogenic wind tunnel simulates flight conditions for scale models—a critical tool in designing airplanes.

3

How about software?

idea → coding

Software Crisis
Poor Quality
(particularly in Taiwan)

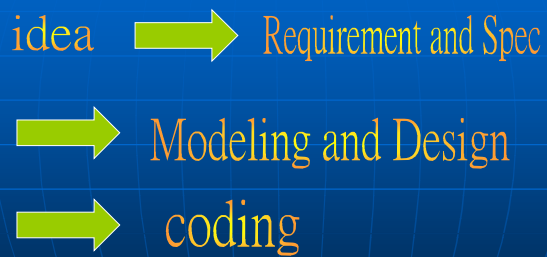
No body has faith in Software

6

3

6

Software Engineering Methodology



7

Types of models

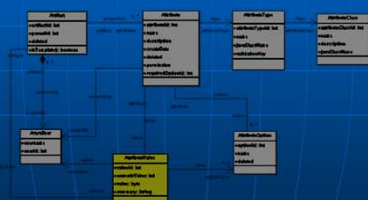
- As in different engineering discipline, there are different kinds of models
- In software development, actually you have use some non-standard models without noticing it.
 - Flow chart
 - Data flow graph
 - Architecture models

10

10

Modeling Technology

- UML (Unified Modeling Language)



8

8

Model classification based on rigidity

- **Model that is not rigid**
 - Some UML models like class diagram
 - ERD models
- **Model that is executable**
 - State chart in UML, can be applied to tools such as simulation
- **Model that is rigid**
 - Can be applied to tools such as verification

11

11

State Chart in UML

- To describe the behaviors of a single object or composite behaviors of objects based on state/event model
- **executable**



9

9

Model classification based on application domain

- **Object-Oriented models (UML)**
- **Concurrency models (finite-state like)**
 - CCS, CSP (Communicating Sequential Process)
 - Petri Net
- **Data models**
 - ERD (entity relationship diagram)

12

12

Modeling process

- Actually, modeling process is a design and analysis process
- Through the process, you will understand, research, and consider your problem carefully. Therefore, avoid fatal error in your problem

13

13

Modeling process

- However, modeling process enforces you to think, research, and understand your problem
- Although your programmers may still build a poor quality code according to your design
- At least, your architecture, design maintained in good shape.
- 也就是說你還保持著你的概念的完整性

16

16

Modeling process

- Modeling process is not omniscient
 - A possible scenario is that
 - You build models
 - You use tools to verify, validate the models and prove your design is correct
 - You have programmers follow your model to implement the system
 - Your system still crash!

14

14

What can you do to a model?

- Check consistency or Proofs
- Translate it into templates, allow programmers to add codes
- Check functional correctness –
 - Simulation – if the type of model is executable
 - Verification – if the type of model is based on some formalism
- Use it as document (Software Visualization)
- Use it as a communicating tools

17

17

However, the scenario is no different in some discipline

- You build models
- You use tools to verify, validate the models and prove your design is correct
- You have your design manufactured
- Your products do not work

15

15

Check consistency and Proofs

- The major risk of design and analysis is omit specification
- There are other tools to prove a model's correctness – manual help is needed, semi-automatically

18

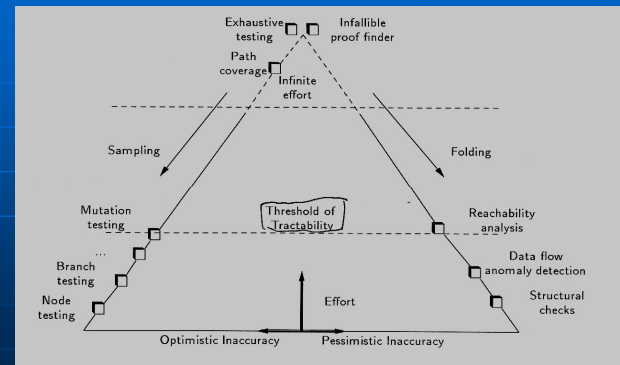
18

Translate a model into templates for coding

- These tools are practical and really used in practice
 - Rationale
 - ArgoUML
 - JBuilder X

19

Optimistic v.s. Pessimistic



22

Check functional correctness

- Consider the EE process again.
 - A model is mostly useful if it can help discover your design flaws in early stage
- Software engineering community always want that !! And want it hard!!!
- They want some models which can be validated or verified

20

Models

- So, we want models can be
 - Simulated!
 - Verified !
- They can provide more clues of how our design is wrong
- By the way, what is design?

23

The basics

- Testing – execution on product or program (implementation) to detect faults (optimistic)
- Simulation – execution on artifacts (models) which is simpler representation of the original complexity (optimistic)
- Verification – explore all the state space of models which is a simpler representation of original complexity (pessimistic)

21

The foundation behind model simulation and verification

$$X \leq Y$$

$$(X > -1) \Rightarrow (Y > -1)$$

$$\begin{array}{ccc} \text{Preorder} & & \\ \text{(implements)} & & \\ \text{Prog} & \leq & \text{Model} \\ \text{deadlock exist} & \Rightarrow & \text{deadlock exists} \end{array}$$

24

19

20

23

21

24